Team Travelling Salesmen

Poker AI · Adaptive Monte Carlo · Adversarial Bidding · Incentive Based Ensembler

Aravind Ramana | Aarya Gosar

Strategy Architecture

BETTING PHASE

- Adaptive Monte Carlo simulation
- Log-based pattern exploitation
- Strength based on heuristics
- Opponent bet tracking
- Bluff detection

BIDDING PHASE

- Pot-Proportional sizing formula
- Vickrey auction limitations identified
- Exploitative pressure increments
- Adversarial cost maximisation
- x1.05 convergence trajectory

COMBINER

- Multi-armed bandit architecture
- ε-greedy exploration policy
- Strategy ensembling (A & B bots)
- Passive vs Aggressive modes
- Decaying epsilon (→ 0.01)

Hand Strength Engine

estimates P(win) via heuristics + simulation — Strength : A designed Proxy for Expected Value of strategies played

PRE-FLOP

O(1) Formula Calculation

1326 starting hands | 133M+ possible games

Estimate Probability heuristically by playing simulated games

O(1) Time complexity

POST-FLOP

Adaptive Monte Carlo Simulation

5 - 8 community cards known, remainder drawn randomly.

Simulates M hands, where M adapts to remaining time budget by maximising accuracy without timing out.

20s Time budget utilized

OPTIMISATION

Hand Result Caching

Repeated hand configurations are looked up from cache rather than recomputed.
Reduces latency on common board states and frees compute for simulation.

O(|A|^d)⟶1 Cache hit cost

Log Analysis & Opponent Modelling

WHERE ARE WE LOSING?

Post-game log inspection identifies rounds, opponents, and street conditions where EV was most negative.

STATISTICAL MODELLING ON LOGS

Logs reveal non optimal bidding strategies followed by opponents.

Inflated Bid patterns observe

FIND PATTERNS & EXPLOIT

Derived heuristics feed directly into bet-sizing and bluff-catching logic for subsequent rounds.

OPPONENT BET TRACKING

- Adjust effective strength according to opponent bets
- Catch bluffs if opponent is constantly betting high
- Winning bid → more confident → take bigger risks
- Statistical baseline of 2-Standard deviations to flag outliers

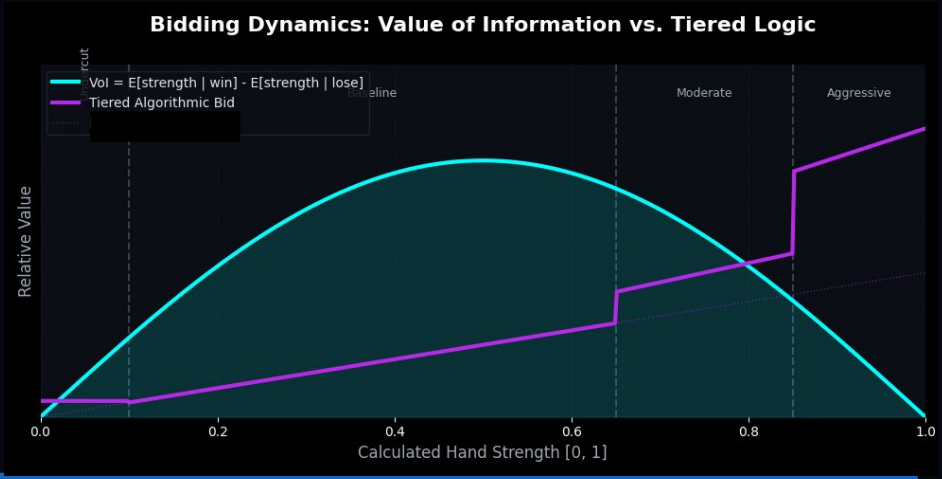
GAUSSIAN- NOISE MODEL

- To Prevent greedy competitors from reverse engineering our bidding strategies from public logs.
- Injected a small Gaussian Noise. The parameters were tuned to be imperceptible to our strategy

Bidding Theory

VICKREY AUCTION: NON-Optimal Bids

- Bid price may enter the pot
- Winning the Bid improves our P(Win the Pot) estimation closer to the actual value
- Escape drawdowns from opponents overbidding and exploiting pot



BID TIER LOGIC

```
if strength < 0.10   : bid = randint(15, 25)          # Undercut strategy for bad hand
elif strength > 0.85 : bid = pot × strength × bid_factor × 3  # Aggressive – very strong
elif strength > 0.65 : bid = pot × strength × bid_factor × 2  # Moderate pressure
else                 : bid = pot × strength × bid_factor × 1.5 # Baseline
```

Adversarial Bid Convergence

THE PROBLEM

Suboptimal bots bid 10×–20× pot, wasting chips and creating extreme price inflation.

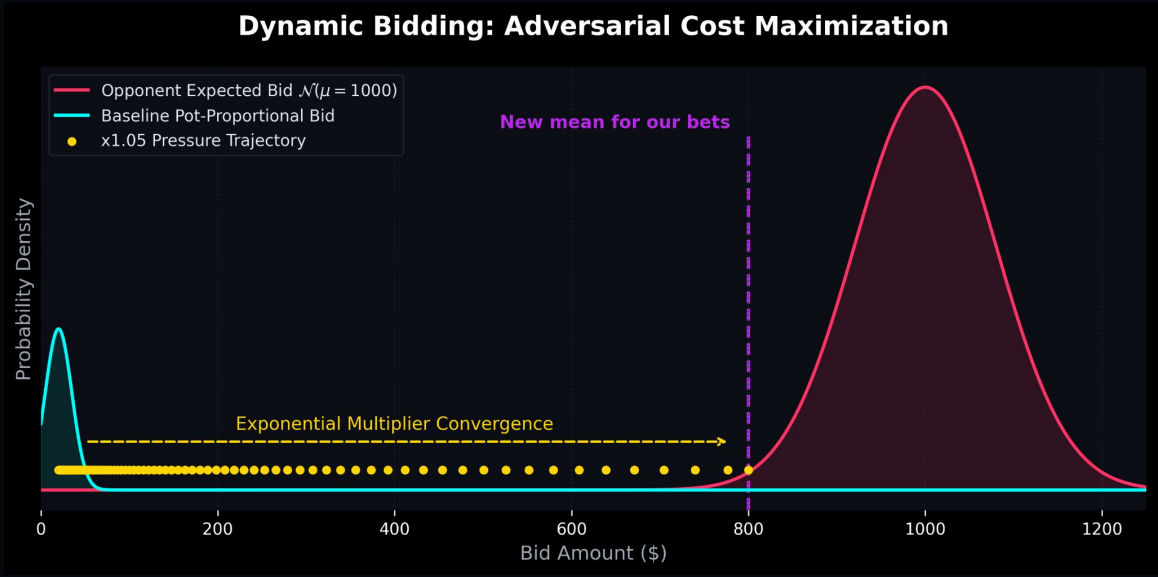
OUR RESPONSE

Incrementally multiply bid - factor by 1.05 when repeatedly losing bids.

Small step but exponentially fast convergence

CONVERGENCE PROPERTY

We slightly underbet our opponents making them pay a high price for their suboptimal bidding logic



Strategy Combiner — Multi-Armed Bandit

BOT A

PASSIVE

Assume Opponent is Safe Player

executing decisions based purely on theoretical Expected Value (EV) and real-time pot odds.

acts as our defensive baseline – securing risk

BOT B

AGGRESSIVE

Assumes the opponent may bluff or play non - optimal.

Assess behaviour tracking their betting distributions

Plays Aggressive exploiting their weak model

SELECTION FLOW

New Hand → Roll Random → ε > r Explore → Pick Bot A/B → Update Q + ε

COMBINER — ε-GREEDY BANDIT

Q-value = net profit accumulated by each bot
ε (epsilon) = exploration probability decaying exponentially with the hands played

```
def on_hand_start(game_info, state):
    if random() < ε:
        bot = random.choice(bots)
    else:
        bot = bots[argmax(Q_values)]

    ε = max(0.01, ε × 0.90)   # decay
```

Exploration horizon: 10–20 games before exploiting optimal bot

Thank You